Lab 1 — Lecture Script (Roman Urdu)

Kaise use karein: Deck A slides project karo. Yeh lines bolo. `// LIVE` = aap machine pe demo.
English terms **bold** rakhein jaisa likha hai.

Block 1 — Hook + outcomes (0:00–0:15) · Slides 1–3

Assalam o alaikum. Yeh **Data Structures Lab** hai — CS-2007L.
Theory course concepts batati hai; yahan hum **implement**, **test**, aur **debug** karte hain.

Aaj ka lab soft nahi hai "sirf Hello World" — aaj foundation hai poore semester ki.

Teen cheezein clear honi chahiye aaj raat tak:

1. **.NET 8** pe project banana, chalana, debug karna
2. **Data, field, record, file** aur **value vs reference**
3. Array pe **six operations** aur chhota **time–space tradeoff**

Real world: university ki **enrolment** list — insert, delete, search, sort.
Factory ke **sensor IDs** — duplicate dhundna.
Pehle tools + array; baad mein linked list, stack, tree, graph.

// Ask: "Kis ne pehle se `dotnet --version` check kiya?" Hands.

Block 2 — Toolchain live (0:15–0:35) · Slides 4–8

Slide 4 dekho.

Aapka code `Program.cs` hai. **Compiler** usay **IL** mein badalta hai — aksar `.dll`.
Phir **CLR** — Common Language Runtime — IL ko **JIT** se machine code banata hai aur chalata hai. Memory **garbage collector** sambhalta hai.

Yaad rakho short line:
Compiler = translate. CLR = run + manage.

// LIVE: terminal
`dotnet --version`
Agar `8.0` se start ho — theek. Warna SDK install / terminal reopen.

Ab project:

```
dotnet new console -n Lab01 --use-program-main
cd Lab01
dotnet run
```

`--use-program-main` is liye: humein clear `Main` method chahiye, top-level statements nahi.

Visual Studio walay: Console App, .NET 8, **Do not use top-level statements**.

`bin` aur `obj` **submit mat karna** — yeh generated folders hain.

// LIVE: Example 1 type / show
Namespace, `Main`, `$"..."`, `///` XML comments.
Output match karo. Version number machine pe alag ho sakta hai — `8.0.x` OK.

Observe: XML comments IDE mein tooltip banate hain — CLO-3 ki shuruaat.

Block 3 — Debugger (0:35–0:55) · Slides 9–11

Teen error types:

1. **Syntax / compile** — build ruk jati hai
2. **Runtime** — chalte waqt exception
3. **Logic** — chal jati hai, jawab galat

Compiler logic error nahi pakarta. Is liye **debugger**.

Keys: **F5** start, **F9** breakpoint, **F10** step over, **Locals** window.

// LIVE: Example 2
`SumToN(5)` expect **15**, milta hai **10**.
Breakpoint `sum += i` pe. `i` aur `sum` dekho.
Loop `i < n` ki wajah se `5` add nahi hota.
Fix: `i <= n`. Dubara run — 15.

Point: "Program happy tha. Bug testing + debugger ne pakra."

Students ko bolo: isi tarah Task 4 karna hai — guess nahi, debugger.

Block 4 — Types + records (0:55–1:15) · Slides 12–15

Terms table:

- **Data** = raw fact — `"Ayesha"`, `3.45`
- **Field** = named attribute — `Name`, `Cgpa`
- **Record** = related fields — ek student
- **File** = records on disk — `students.txt`

Ab **value vs reference** — board pe boxes.

`int`: assign = value copy. `b` change → `a` safe.
`int[]`: assign = **reference** copy. Dono same heap object.
`Clone()` = independent copy.
`string` reference hai lekin **immutable** — `+=` naya string.

// LIVE: Example 3 highlights (poora type karne ka time na ho to aap run karke dikhao)
Overflow: `int.MaxValue + 1` wrap — unchecked.

// LIVE: Example 4
`StudentRecord` → CSV line → file write/read.
Notepad mein `students.txt` khol ke dikhao — "yeh file hai."

Block 5 — Six ops (1:15–1:35) · Slides 16–18

Array pe classic ops: traverse, search, insert, delete, sort, merge.

Costs (intuition, Lab 2 formal):

- Index read ≈ **O(1)**

- Linear search / traverse ≈ **O(n)**
- Insert/delete ≈ **O(n)** kyunki shift/copy
- Merge sorted ≈ **O(n+m)**

// LIVE: Example 5 walk output
Insert @2, delete index 0, sort, merge.
Warning: Merge **sorted** inputs maanta hai.

Challenge: `Insert` galat index → exception → `try/catch` idea.

Block 6 — Tradeoff (1:35–1:50) · Slides 19–20

Kabhi algorithm **tez** chahiye → zyada memory.
Kabhi memory kam → time accept.

// LIVE: Example 6
20,000 queries. Linear vs **HashSet**.
`dotnet run -c Release` — Debug timing confuse karti hai.

Dono same count find karein. HashSet tez; cost = extra space.

Pocho: "Cloud server pe kaun? Edge device pe kaun?"
Yahi Task 6 ka dimagh hai.

Block 7 — Tasks launch (1:50–2:40) · Slide 21

Ab graded work. Priority: **Tasks 1–4** lab mein.

1. Environment report + screenshots + 2 short answers
2. Profile card — `TryParse` loop, scholarship CGPA ≥ 3.00
3. Types table + written classification + reference question
4. `BuggyStats` — **teen bugs**, debugger screenshots, corrected class

Tasks 5–6 mushkil hain — time ho to start; warna homework.

Main ghoomunga. Task 4 pe debugger dikhana zaroori hai.

// Circulate using Deck B hint ladder

Block 8 — Wrap (2:40–2:55) · Slide 22

Submission: Name, Reg No, screenshots code+output, Task 4 debugger shots, PDF LMS.
`bin` / `obj` nahi.
AI use → disclosure template Section 8.4. Agar AI nahi use kiya to likho **None**.

Post-lab Q1–Q3 Assignment 1 ka hissa.
Quiz 1 Labs 1–3 cover karega.

Next week: **Big-O**, frequency count, measured growth — Lab 2.

Block 9 — Exit ticket (2:55–3:00) · Deck C

Random 1–2 questions (e.g. Q1 + Q4, or Q6 + Q13).
Short answers; marks viva component se align.

Shukriya — code khud type karo, Observe notes padho, AI hide mat karna.

Backup lines (agar time short ho)

Agar 20 minutes peeche:

- Example 3/4 mein se ek skip — slides pe theory, demo chhota
- Example 6 sirf aap run karke numbers dikhao
- Tasks 5–6 fully homework declare

Kabhi mat skip: debugger live + Tasks 1–4 launch.